

Quantum information and quantum computation homework 1

Zhouyue Qian 3220104074 *

(Information and Computational Science Class 2201), Zhejiang University

Due time: October 15, 2024

1 Turing Machine 1

I have built a Turing machine that can reverse the input string. It can reverse the input composed of "0"s and "1"s, similar to how "1001" would become "0110".

1.1 Design

1.1.1 Symbols and States

- **External Alphabet A :** $\{0, 1\}$
- **Tape Alphabet S :** $\{_, 0, 1, *\}$, where $*$ is an auxiliary marker.
- **State Set Q :** $\{q_0, q_f, r, l, q_{Swap}, q_{Back}, q_{Skip}\}$, where:
 - r indicates moving right.
 - l indicates moving left.
 - q_{Swap} indicates the swapping phase.
 - q_{Back} indicates returning to the leftmost position after swapping.
 - q_{Skip} indicates skipping consecutive blanks.
 - q_f is the final state.

1.1.2 Transition Function

1. **Initial Phase (q_0):** Mark the leftmost symbol and prepare to move right to find the rightmost symbol.

$$(q_0, 0) \rightarrow (r, *, +1)$$

$$(q_0, 1) \rightarrow (r, *, +1)$$

$$(q_0, _) \rightarrow (q_{Skip}, _, +1)$$

2. **Skipping Consecutive Blanks (q_{Skip}):** Skip consecutive blanks until a non-blank symbol is found.

$$(q_{Skip}, _) \rightarrow (q_{Skip}, _, +1)$$

$$(q_{Skip}, 0) \rightarrow (r, 0, +1)$$

$$(q_{Skip}, 1) \rightarrow (r, 1, +1)$$

*Electronic address: 3220104074@edu.zju.cn

3. **Moving Right (r):** Find the rightmost unprocessed symbol.

$$(r, 0) \rightarrow (r, 0, +1)$$

$$(r, 1) \rightarrow (r, 1, +1)$$

$$(r, _) \rightarrow (q_{Swap}, _, -1)$$

4. **Swapping Phase (q_{Swap}):** Move left to find the marker $*$ and perform the symbol swap.

$$(q_{Swap}, 0) \rightarrow (q_{Swap}, 0, -1)$$

$$(q_{Swap}, 1) \rightarrow (q_{Swap}, 1, -1)$$

$$(q_{Swap}, *) \rightarrow (q_{Back}, _, +1)$$

5. **Returning to the Start (q_{Back}):** Return to the leftmost position to process the next symbol.

$$(q_{Back}, 0) \rightarrow (q_{Back}, 0, +1)$$

$$(q_{Back}, 1) \rightarrow (q_{Back}, 1, +1)$$

$$(q_{Back}, *) \rightarrow (r, *, +1)$$

6. **Final State (q_f):** When all symbols are processed, stop.

$$(r, *) \rightarrow (q_f, *, 0)$$

1.2 Advanced Analysis

This Turing machine that can reverse an input binary string and handle consecutive blank spaces.

1.2.1 Time Complexity

- **Initial Phase (q_0):** $O(1)$
- **Moving Right (r):** $O(n)$
- **Swapping Phase (q_{Swap}):** $O(n)$
- **Returning to the Start (q_{Back}):** $O(n)$
- **Final State (q_f):** $O(1)$

Total Time Complexity: $O(n^2)$, where n is the length of the input string.

1.2.2 Space Complexity

- **Initial Phase (q_0):** $O(1)$
- **Moving Right (r):** $O(1)$
- **Swapping Phase (q_{Swap}):** $O(1)$
- **Returning to the Start (q_{Back}):** $O(1)$
- **Final State (q_f):** $O(1)$

Total Space Complexity: $O(n)$, where n is the length of the input string.

2 Turing Machine 2

This Turing machine can add two numbers written in binary.

2.1 States

- q_0 : Initial state. Reads the input and identifies two binary numbers separated by a delimiter "+".
- q_1 : Scanning the first binary number.
- q_2 : Carrying out the addition of corresponding bits from the two numbers.
- q_3 : Writing the result and handling the carry.
- q_4 : Final carry handling.
- q_f : Final state after completion.

2.2 External Alphabet

- 0, 1: Binary digits.
- +: Delimiter separating two numbers.
- $_$: Blank symbol.

2.3 State Transition Function

Current State	Read Symbol	Write Symbol	Move Direction	Next State
q_0	0	0	R	q_0
q_0	1	1	R	q_0
q_0	+	+	R	q_1
q_1	0	0	R	q_1
q_1	1	1	R	q_1
q_1	$_$	$_$	L	q_2
q_2	0 (no carry)	0	L	q_3
q_2	1 (no carry)	1	L	q_3
q_2	1 + 1 (no carry)	0	L	q_3 (carry)
q_3	0	0	L	q_3
q_3	1	1	L	q_3
q_3	+	+	L	q_4
q_4	No carry	$_$	-	q_f
q_4	Carry	1	L	q_f

Table 1: State Transition Table for Binary Addition

2.4 Explanation of the Process

1. In the initial state q_0 , the Turing Machine scans the first binary number and stops when it encounters the "+" delimiter. 2. Once the "+" symbol is detected, the machine enters state q_1 and begins searching for the rightmost bit of the second binary number. 3. In state q_2 , the machine adds the binary digits bit by bit, starting from the rightmost bit of both numbers. 4. The machine handles carry bits accordingly. If there is a carry after adding the bits, it enters state q_3 to continue the process. 5. In state q_3 , the machine writes the result and moves left to handle the next bit or the carry. 6. If a final carry exists after all bits have been processed, the machine enters state q_4 and writes the carry before reaching the final state q_f . 7. The Turing Machine halts in state q_f once all operations are complete.

2.5 Example: Adding 101 + 11

For the input "101+11", the Turing Machine will proceed as follows:

- Initial state q_0 : Read "101", then encounter "+" and move to state q_1 .
- State q_1 : Start reading "11" from right to left.
- State q_2 : Perform the addition:
 - $1 + 1 = 10$: Write 0, carry 1.
 - $0 + 1 + 1 = 10$: Write 0, carry 1.
 - $1 + 0 + 1 = 10$: Write 0, carry 1.
- State q_4 : Write the final carry, resulting in "1000".

Thus, the binary addition of 101 and 11 yields the result 1000 (which equals 8 in decimal).

3 Turing Machine proof

3.1 Basic Idea of the Simulation

The basic idea of simulating a 2-tape Turing Machine M_2 with a single-tape Turing Machine M_1 involves encoding the two tapes of M_2 on a single tape of M_1 . To do this, we need to:

- Use an expanded alphabet for M_1 that can encode the symbols on both tapes of M_2 , along with markers for the head positions.
- Simulate each step of M_2 by scanning the entire tape of M_1 to find the head positions, reading the symbols under each head, and then updating the tape according to the transition function of M_2 .

3.1.1 1-Tape Alphabet

The alphabet of M_1 must be large enough to encode the symbols of both tapes of M_2 . Let Σ be the alphabet of M_2 , including the blank symbol _____. Then, each cell of M_1 's tape must encode:

- A symbol from Σ for tape 1 of M_2 .
- A symbol from Σ for tape 2 of M_2 .

- Two additional bits to mark the head positions on each of the two tapes.

Thus, the alphabet of M_1 is $\Sigma^2 \times \{0,1\}^2$, which includes all possible combinations of symbols from both tapes and head markers.

3.1.2 Initial Configuration

Initially, the input on the first tape of M_2 is encoded on the single tape of M_1 . The second tape of M_2 is initialized to be blank. The head positions of both tapes of M_2 are marked in the encoding on the single tape of M_1 .

3.2 Simulation Process

The simulation proceeds by simulating each step of M_2 in two phases:

1. A left-to-right scan of the tape to locate the head positions and read the symbols under both heads of M_2 .
2. A right-to-left scan to perform the actions dictated by the transition function of M_2 (write symbols and move the heads).

3.2.1 Phase 1: Left-to-Right Scan

In this phase, M_1 performs a left-to-right scan of the entire tape to find the head positions of both tapes of M_2 . During this scan, M_1 stores the symbols found under the heads of M_2 in its finite control. This takes $O(s)$ steps, where s is the length of the portion of the tape that contains non-blank symbols.

3.2.2 Phase 2: Right-to-Left Scan

After reading the symbols under both heads in Phase 1, M_1 performs a right-to-left scan of the tape. During this scan, M_1 updates the symbols on the tape according to the transition function of M_2 , moves the head markers, and performs any necessary write operations. This also takes $O(s)$ steps.

Since each phase takes $O(s)$ steps, the total time for simulating one step of M_2 is $O(s)$.

3.3 Time Complexity Analysis

3.3.1 Tape Length

Let s denote the length of the portion of the tape used by M_2 . In the worst case, the tape length grows with the number of steps $T(n)$, so $s \leq T(n) + 1$.

3.3.2 Total Time Complexity

Each simulated step of M_2 requires $O(s)$ steps by M_1 . Since M_2 runs for $T(n)$ steps, the total time required by M_1 to simulate all steps of M_2 is:

$$O(s \cdot T(n)) = O(T(n) \cdot T(n)) = O(T(n)^2).$$

Thus, the total time complexity of simulating a 2-tape Turing Machine with a single-tape Turing Machine is $O(T(n)^2)$. □